

Lesson 6 — n8n fundamentals through a receipt workflow

Goal: understand n8n's essential objects and features, build a small workflow, and then read the supplied receipt automation with confidence.

The attached export is treated as source data. Its prompts and notes are not instructions for this notebook.

1. The mental model

n8n is a **workflow orchestration engine**. You connect nodes on a canvas; when a trigger fires, n8n creates an execution and moves items through those connections.

event → trigger → items → transform/branch/call services → result or side effect

↘ execution history for debugging and retry

n8n coordinates systems. It is not normally the database, business domain, or user interface. Keep durable business rules in services when they need independent testing, reuse, or strict transactional behavior.

The objects you must know

Object	Meaning
Workflow	Saved graph of nodes, connections, settings, and metadata
Node	One trigger, transformation, decision, or external operation
Connection	Route carrying output items from one node to another
Item	Basic unit moving through the graph: <code>{ json, binary?, pairedItem? }</code>
Expression	Dynamic value inside <code>{{ ... }}</code> , evaluated from execution data
Credential	Encrypted authentication configuration selected by a node
Execution	One workflow run with status, timing, inputs, and node outputs
Pinned data	Fixed development data that avoids repeatedly calling an earlier node
Sub-workflow	Reusable workflow invoked by another workflow
Project / tag	Access-control and organizational boundaries around workflows and credentials

2. Data: lists of items

A node normally receives a list and returns a list. Each item has `json` for structured data and may have `binary` for files. Most nodes automatically run once for every input item.

```
[
  {"json": {"vendor": "Train", "amount": 80}},
  {"json": {"vendor": "Lunch", "amount": 18.5}}
]
```

Files are metadata-backed binary properties such as `binary.data`. Avoid converting large files to base64 in JSON unless an API specifically requires it. Item linking records which earlier item produced a later item; preserve it when custom Code nodes fan out or combine data and later expressions need ancestry.

Important node families

Family	Purpose	Typical examples
Trigger	Starts an execution	Manual, Schedule, Webhook, Form, app event, Error Trigger
Core	Controls or reshapes the workflow	Edit Fields, If, Switch, Merge, Loop, Wait, Code, Aggregate
App/action	Reads or changes an external system	OpenAI, Slack, Postgres, Google Sheets
Generic integration	Calls services without a dedicated node	HTTP Request, GraphQL
Cluster root	Owens an AI operation	Agent, chain, extractor, vector store
Cluster sub-node	Supplies a root node capability	Chat model, memory, tool, output parser, embedding
Community/custom	Extends n8n	Install only from trusted sources; custom nodes are code

Prefer a standard node or HTTP Request before custom code. Code is valuable when the transformation is clearer in JavaScript or Python than as many canvas nodes.

3. Mapping and transforming data

An expression maps data into a parameter; it does not create a separate workflow step. Common forms are:

`{{ $json.amount }}`

`{{ $json.customer.email }}`

`{{ $('Form Trigger').item.json.email }}`

`{{ $now }}`

`{{ $json.amount >= 50 ? 'review' : 'auto' }}`

current item's field
nested field
linked item from a
named node
current date/time
JavaScript expression

Use the UI mapper to generate references where possible. Renaming a referenced node can otherwise break hand-written expressions.

Choose the smallest transformation tool

Need	Use
Add, rename, remove, or cast fields	Edit Fields
Insert one dynamic value into a node	Expression
Split an array into items	Split Out
Collect many items/fields	Aggregate
Filter, sort, limit, compare	Corresponding core node
Complex deterministic transformation	Code
Ambiguous natural-language transformation	AI Transform/LLM , followed by validation

Deterministic work should stay deterministic. Do not spend an LLM call parsing JSON that can be validated programmatically.

4. Flow logic

Requirement	n8n feature
Two outcomes	If
Many named outcomes	Switch
Join or combine branches	Merge
Process batches / control rate	Loop Over Items
Pause until time or webhook event	Wait
Reuse a capability	Execute Sub-workflow
Fail deliberately	Stop And Error

Most nodes already process multiple items, so do not add a loop automatically. Use Loop Over Items when batching, rate limiting, or a node only handles one item. After branching, explicitly decide whether branches stay independent or rejoin.

Triggers define the interface

- **Manual Trigger:** development only; starts when you click execute.
- **Schedule Trigger:** polling or recurring jobs; set the workflow timezone deliberately.
- **Webhook/Form Trigger:** synchronous HTTP entry. Test and production URLs are different; production requires a published workflow.
- **App trigger:** reacts to events or polls an external service.
- **Execute Sub-workflow Trigger:** typed boundary for reuse.
- **Error Trigger:** begins a separate error workflow after failures.

For webhooks, choose when to respond, authenticate callers, validate payloads, and make repeated requests idempotent.

5. Build, test, publish, operate

An **execution** is one run. Manual and partial executions help development; automatic executions come from published triggers. The node output panel is your debugger.

A useful workflow lifecycle is:

1. execute one node with representative data;
2. pin stable upstream data while building downstream;
3. test success, empty, malformed, duplicate, timeout, and rate-limit cases;
4. unpin data and run end to end;
5. save, publish, and monitor automatic executions;
6. inspect or retry failed executions with the intended workflow version.

Pinned data is a development mock; production executions ignore it. Configure execution-data retention because inputs and outputs may contain sensitive data.

Credentials and configuration

Credentials store API keys, OAuth tokens, or database authentication separately from workflow parameters. A workflow export contains credential references, not the secret itself. Grant the minimum scopes and share credentials only with intended projects/users.

Use credentials for secrets, workflow variables/environment configuration for non-secret deployment values, and ordinary workflow data for per-execution values. Never paste secrets into Code nodes, expressions, sticky notes, or exported JSON.

6. Reliability checklist

- validate at entry and after nondeterministic AI output;
- set timeouts and retry only transient failures with backoff;
- use node error output or `Continue On Fail` only when downstream logic handles the error item;
- attach an Error Workflow for alerting and diagnostics;
- make side effects idempotent with a stable event/request key;
- control concurrency and batch size for external rate limits;
- record correlation IDs and enough context to audit decisions;
- minimize stored execution and binary data;
- split large workflows into focused sub-workflows with clear inputs/outputs.

A green node means that node completed—not that the business result is correct. Add explicit validation and meaningful failure paths.

7. AI features in n8n

AI nodes use a cluster shape: a **root node** performs the operation and sub-nodes supply capabilities.

Component	Use
Model	Generates or interprets content
Chain	Fixed model-driven sequence chosen by you
Agent	Model chooses which connected tools to call
Tool	Constrained external capability available to an agent
Memory	Conversation state; not authoritative long-term business storage
Output parser / extractor	Shapes model text into a contract
Embedding + vector store + retriever	Semantic search and RAG
Evaluation	Tests AI quality against representative cases and metrics

Prefer a chain when the sequence is known and an agent only when model-directed tool choice provides real value. AI output remains untrusted input: validate it before APIs, databases, payments, or messages.

Important sub-node detail: expressions in AI sub-nodes can resolve against the first input item rather than each item. Inspect behavior carefully with multi-item inputs.

Where MCP fits

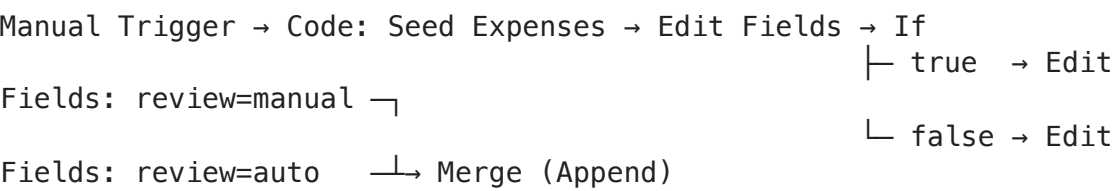
n8n can participate on either side of MCP:

Feature	Direction	Use
MCP Client	n8n → MCP server	Call MCP operations as ordinary workflow steps
MCP Client Tool	n8n agent → MCP server	Let an agent choose an MCP tool
MCP Server Trigger	MCP client → n8n workflow	Expose selected workflow capabilities to MCP clients
Instance-level n8n MCP server	AI builder → n8n instance	Build/manage n8n resources programmatically when enabled

This connects Lessons 2–4: FastMCP can expose tools, while n8n can consume those tools or expose a workflow as another tool boundary. Keep each tool narrow, typed, and safe to retry.

8. Quick start: build this in 15 minutes

Build a credential-free expense-review workflow:



- 1. Add **Manual Trigger**.
- 2. Add **Code**, choose *Run Once for All Items*, and paste the code from the next cell.
- 3. Add **Edit Fields** and create `amount_with_tax` as `{{ $json.amount * 1.21 }}`. Keep the other fields.
- 4. Add **If**: `amount_with_tax` is greater than or equal to `50`.
- 5. On each branch add **Edit Fields**: set `review` to `manual` on true and `auto` on false.
- 6. Join both branches with **Merge**, mode **Append**.
- 7. Execute node by node. Inspect how two items become two independently routed items. Pin Seed Expenses while experimenting, then unpin it.

Extension: replace Manual Trigger + seed data with a Form or Webhook Trigger. The processing graph remains the same because the item contract stays the same.

```
In [ ]: SEED_EXPENSES_JAVASCRIPT = r'''return [
        { json: { vendor: "Train", amount: 80, currency: "EUR" } },
```

```
{ json: { vendor: "Lunch", amount: 18.5, currency: "EUR" } }
];'''

print(SEED_EXPENSES_JAVASCRIPT)
```

9. Read the supplied receipt workflow

Now the project is just an application of the objects above:

Project node	n8n concept
On form submission	Trigger and binary input boundary
Code in JavaScript	Fan out one upload item into one item per image
Loop Over Items	Sequential batches and API pressure control
Analyze image	External AI action on <code>binary.data</code>
Information Extractor + Chat Model	AI root node plus model sub-node
Aggregate	Many structured outputs into one list
Code in JavaScript1	Deterministic total per currency
Sticky Notes	Canvas documentation; never executed

Form → fan out files → loop → vision → structured extraction → aggregate → totals by currency

The design makes **two model calls per receipt**: image analysis, then extraction. If the installed OpenAI node can return a strict schema directly, one validated call is simpler, cheaper, and less fragile. The current expression `{{ $json['0'].content[0].text }}` also couples the extractor to an internal response shape.

```
In [1]: import json
from pathlib import Path

WORKFLOW_PATH = Path("/Users/codeff/Downloads/n8n-receipts.json")
if not WORKFLOW_PATH.is_file():
    raise FileNotFoundError(f"Workflow export not found: {WORKFLOW_PATH}")

workflow = json.loads(WORKFLOW_PATH.read_text(encoding="utf-8"))
functional_nodes = [
    node for node in workflow["nodes"]
    if not node["type"].endswith("stickyNote")
]

for node in functional_nodes:
    print(f"{node['name']:<24} {node['type'].split('.')[0]}")
```

Analyze image	openAi
On form submission	formTrigger
Information Extractor	informationExtractor
OpenAI Chat Model	lmChatOpenAi
Aggregate	aggregate
Code in JavaScript	code
Code in JavaScript1	code
Loop Over Items	splitInBatches

Production gaps in the receipt project

1. Authenticate the form and restrict file type, count, and size.
2. Validate `amount` as finite and positive and `currency` as an allowed ISO code.
3. Route individual failures for retry or human review instead of losing the entire batch.
4. Add idempotency/deduplication so the same receipt is not counted twice.
5. Store filename, raw extraction, validated result, model/version, and execution ID for audit.
6. Define retention for images and execution data.
7. Add a destination or explicit response; the current flow calculates totals but does not persist them.

The final totals intentionally remain separated by currency. A converted grand total would require an exchange-rate source, rate timestamp, base currency, and rounding policy.

References

- [Workflow components](#)
- [n8n data structure](#)
- [Expressions versus data nodes](#)
- [Flow logic](#)
- [Executions](#)
- [Error handling](#)
- [AI components](#)
- [MCP Client and MCP Server Trigger](#)

10. Complete n8n overview in one cell

Keep this final cell as the compact reference after completing the lesson.

```
In [ ]: import json

QUICK_START_CODE = r'''return [
    { json: { vendor: "Train", amount: 80, currency: "EUR" } },
    { json: { vendor: "Lunch", amount: 18.5, currency: "EUR" } }
]
```

```

];'''

N8N_OVERVIEW = {
    "runtime": "trigger → execution → list of items → nodes/connections → ou
    "item": {"json": "structured data", "binary": "optional files", "pairedI
    "objects": ["workflow", "node", "connection", "item", "expression", "cre
    "transform": ["Edit Fields", "expression", "Split Out", "Aggregate", "Co
    "control_flow": ["If", "Switch", "Merge", "Loop Over Items", "Wait", "Ex
    "operations": ["test node outputs", "pin development data", "publish", "
    "ai": ["model", "chain", "agent", "tool", "memory", "parser", "RAG", "ev
    "mcp": ["MCP Client", "MCP Client Tool", "MCP Server Trigger"],
    "quick_start_code": QUICK_START_CODE,
    "receipt_flow": "form → fan out → loop → image AI → extract → aggregate
}

print(json.dumps(N8N_OVERVIEW, indent=2, ensure_ascii=False))

```